Tests de errores y respuestas HTTP coherentes

José MSA

`docs/Cap02bis_Tests_Tasks_Errores.md`

Capítulo 2 bis — Tests de errores y payload consistente (Tasks)

Módulo: DWES
Capítulos: 2 bis (Tests)
RA relacionados: RA1, RA5

Objetivo

Asegurar que la API devuelve **errores coherentes y uniformes** (códigos + cuerpo JSON) usando `@ControllerAdvice` y probarlos con **MockMvc**.

1. Handler global de errores (`@ControllerAdvice`)

Creamos un payload de error uniforme: `{ error, message, timestamp }`.

```java
// src/main/java/com/example/tasks/web/GlobalExceptionHandler.java
package com.example.tasks.web;

import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.*;
import java.time.Instant;
import java.util.Map;

@ControllerAdvice
public class GlobalExceptionHandler {

  @ExceptionHandler(MethodArgumentNotValidException.class)
  public ResponseEntity<?> validation(MethodArgumentNotValidException ex) {
    String msg = ex.getBindingResult().getAllErrors().get(0).getDefaultMessage();
    return ResponseEntity.badRequest().body(Map.of(
      "error", "VALIDATION_ERROR",
      "message", msg,
      "timestamp", Instant.now().toString()
    ));
  }

  @ExceptionHandler(IllegalArgumentException.class)
  public ResponseEntity<?> badRequest(IllegalArgumentException ex) {
    return ResponseEntity.badRequest().body(Map.of(
      "error", "BAD_REQUEST",
```

```
      "message", ex.getMessage(),
      "timestamp", Instant.now().toString()
    ));
  }

  @ExceptionHandler(Exception.class)
  public ResponseEntity<?> generic(Exception ex) {
    return ResponseEntity.status(500).body(Map.of(
      "error", "INTERNAL_ERROR",
      "message", "Unexpected error",
      "timestamp", Instant.now().toString()
    ));
  }
}
```

Qué probamos: que una validación fallida devuelva **400** con `error=VALIDATION_ERROR`, y que errores de lógica devuelvan **400** con `error=BAD_REQUEST`.

2. Ajustes mínimos del controlador para provocar errores

- Validación en `POST` (ya añadida con `@Valid` en `CreateTaskDTO`).
- Lógica de error en `GET` por id inexistente, si preferimos lanzar excepción (opcional):

```
@GetMapping("/{id}")
public ResponseEntity<Task> get(@PathVariable Long id) {
  return tasks.stream()
      .filter(t -> t.getId().equals(id))
      .findFirst()
      .map(ResponseEntity::ok)
      .orElseThrow(() -> new IllegalArgumentException("Task not found"));
}
```

(si prefieres 404 sin excepción, ajusta el test a `notFound()` y omite este handler para 404; aquí lo testeamos por coherencia del ejemplo con `BAD_REQUEST`)

3. Tests con MockMvc: 400 validación y 400 lógica

```
// src/test/java/com/example/tasks/web/TaskErrorsTest.java
package com.example.tasks.web;

import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.webmvc.test.autoconfigure.WebMvcTest;
import org.springframework.http.MediaType;
import org.springframework.test.web.servlet.MockMvc;

import static org.hamcrest.Matchers.*;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.*;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;

@WebMvcTest({ TaskController.class, GlobalExceptionHandler.class })
class TaskErrorsTest {

  @Autowired MockMvc mvc;

  @Test
  void create_shouldReturnValidationError_whenTitleBlank() throws Exception {
```

```
    String invalid = """{ "title": "   " }""";

    mvc.perform(post("/api/tasks")
            .contentType(MediaType.APPLICATION_JSON)
            .content(invalid))
       .andExpect(status().isBadRequest())
       .andExpect(content().contentTypeCompatibleWith(MediaType.APPLICATION_JSON))
       .andExpect(jsonPath("$.error").value("VALIDATION_ERROR"))
       .andExpect(jsonPath("$.message", containsString("must not be blank")))
       .andExpect(jsonPath("$.timestamp", not(emptyOrNullString())));
  }

  @Test
  void get_shouldReturnBadRequestPayload_whenIllegalArgument() throws Exception {
    mvc.perform(get("/api/tasks/9999"))
       .andExpect(status().isBadRequest())
       .andExpect(jsonPath("$.error").value("BAD_REQUEST"))
       .andExpect(jsonPath("$.message", containsString("Task not found")))
       .andExpect(jsonPath("$.timestamp", not(emptyOrNullString())));
  }
}
```

4. Tests de contenido no soportado (415) y método no permitido (405)

Estos dos estados los gestiona Spring MVC automáticamente; verificamos **código** y opcionalmente el **Content-Type**.

```
@Test
void create_shouldReturn415_whenWrongContentType() throws Exception {
  mvc.perform(post("/api/tasks")
          .contentType(MediaType.TEXT_PLAIN) // incorrecto
          .content("title=texto"))
     .andExpect(status().isUnsupportedMediaType()); // 415
}

@Test
void tasks_shouldReturn405_whenMethodNotAllowed() throws Exception {
  mvc.perform(put("/api/tasks")) // no hay PUT definido
     .andExpect(status().isMethodNotAllowed()); // 405
}
```

5. Checklist de calidad del error payload

- ☐ Siempre JSON con `{ error, message, timestamp }`.
- ☐ `400` validación → `error = VALIDATION_ERROR`.
- ☐ `400` lógica → `error = BAD_REQUEST` + mensaje entendible.
- ☐ No filtrar **detalles internos** en producción (log sí, respuesta no).
- ☐ Añadir `traceId` o `correlationId` (opcional) más adelante.

6. Resumen docente

- Con `@ControllerAdvice` centralizamos los errores.
- Probamos **codificación HTTP** y **shape del payload**.
- Deja la API lista para frontends y para depurar incidencias.
